

IoT Hardware Integration Handoff Report

Project: Vision-Based Smart Energy Saving System (v2.0)

Target Audience: Hardware Engineers, Embedded/IoT Developers, Network Admins

Document Version: 2.1.0

The Judgment & Critique

What You Did Well

- Clear Decoupling: Separating the Vision Master from the IoT Subscriber layer makes the architecture pluggable and modular.
- Appropriate Relay Selection: Sizing a 30A SSR for a 1200W AC unit versus 10A mechanical relays for lighting/fans shows proper load considerations.
- Dual Integration Strategy: Providing both an in-process Python adapter and an out-of-process WebSocket interface gives downstream engineers immediate flexibility.

Where It Falls Short for Hardware / Embedded Engineers

1. Vague MQTT Architecture: You mention MQTT in text, but give zero topic topologies (``telemetry/``, ``cmd/``, ``status/``). Embedded developers need explicit topic definitions and QoS levels.
2. ESP32 Boot Pin Traps: Recommending GPIO pins without specifying boot behavior is risky. On the ESP32, several pins pull HIGH at boot or act as strapping pins, causing relays to chatter or the MCU to fail to boot.
3. No Hardware Protection Circuitry: Missing critical electrical safety specs: flyback diodes for inductive fan loads, RC snubbers for AC switching, and optocoupler active-LOW/HIGH logic.
4. Vision Bouncing / Flickering: No mention of Vision Hysteresis / Debounce time. Raw vision inference flickers; without delay/debounce parameters, physical relays will chatter continuously, burning out contact points.
5. Fail-Safe / Contact Logic: Missing explicit Normal Open (NO) vs. Normal Closed (NC) guidance during power loss or system failure.

Enhanced Integration Handoff Report

Below is the upgraded, industry-standard handoff document designed for seamless handoff to hardware, embedded, and network engineers.

IOT HARDWARE INTEGRATION HANDOFF REPORT

Project: Vision-Based Smart Energy Saving System (v2.0)

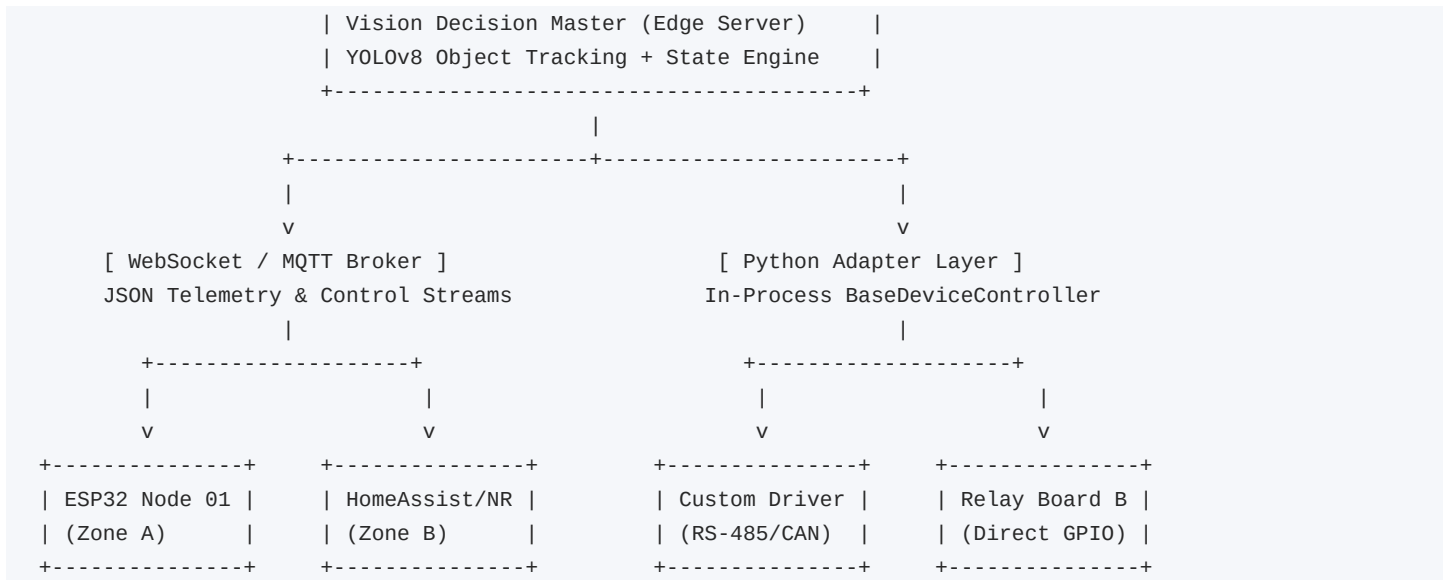
Target Audience: Hardware Engineers, Embedded/IoT Developers, Network Admins

Document Version: 2.1.0

1. System Architecture & Topology

The Vision Decision Master acts as a centralized edge engine running inference (YOLOv8 + temporal state smoothing). It evaluates room occupancy and broadcasts target device states to distributed IoT Actuators / Relays (ESP32, Home Assistant, Node-RED).

+-----+



2. Communication Interfaces & API Contracts

Option A: In-Process Python Adapter Pattern

Located at `app/device_controller/base.py`. Developers extending the core backend directly in Python must subclass `BaseDeviceController`:

```

from app.device_controller.base import BaseDeviceController
import requests
import logging

logger = logging.getLogger(__name__)

class HomeAssistantAdapter(BaseDeviceController):
    """Adapter for syncing vision states with Home Assistant REST API."""

    def __init__(self, host: str, token: str):
        self.base_url = f"http://{host}:8123/api/services/switch"
        self.headers = {
            "Authorization": f"Bearer {token}",
            "Content-Type": "application/json",
        }

    def set_device_state(self, device_id: str, state: str, reason: str = "") -> bool:
        action = "turn_on" if state.upper() == "ON" else "turn_off"
        url = f"{self.base_url}/{action}"
        payload = {"entity_id": f"switch.{device_id}"}

        try:
            response = requests.post(url, json=payload, headers=self.headers, timeout=3.0)
            response.raise_for_status()
            logger.info(f"[HA Adapter] {device_id} -> {state} ({reason})")
            return True
        except requests.RequestException as err:
            logger.error(f"[HA Adapter Failed] {device_id}: {err}")
            return False
  
```

Option B: MQTT Protocol Spec (Recommended for ESP32 / Bare-Metal)

- Broker Port: `1883` (Unencrypted) / `8883` (MQTTS - TLS 1.3)
- QoS Level: `1` (At least once)
- Retain Flag: `True` for state updates

Topic Structure:

- State Broadcast (Server -> IoT): `v2/energy/zone\_a/devices/state`
- Command Override (IoT -> Server): `v2/energy/zone\_a/devices/command`
- Node Telemetry / LWT (IoT -> Server): `v2/energy/nodes/{client\_id}/telemetry`

Inbound State Broadcast Payload:

```
{
  "timestamp": 1754783162,
  "occupant_count": 1,
  "occupancy_status": "OCCUPIED",
  "device_states": {
    "light": "ON",
    "fan": "ON",
    "ac": "OFF"
  },
  "device_telemetry": {
    "light": { "state": "ON", "power_pct": 100.0, "is_ramping": false },
    "fan": { "state": "ON", "power_pct": 100.0, "is_ramping": false },
    "ac": { "state": "OFF", "power_pct": 0.0, "is_ramping": false }
  }
}
```

Option C: WebSocket API Contract

- URL: `ws://<SERVER\_IP>:8000/ws/status`
- Ping / Pong Interval: Every 10 seconds.

3. Recommended Hardware Specifications & Pinouts

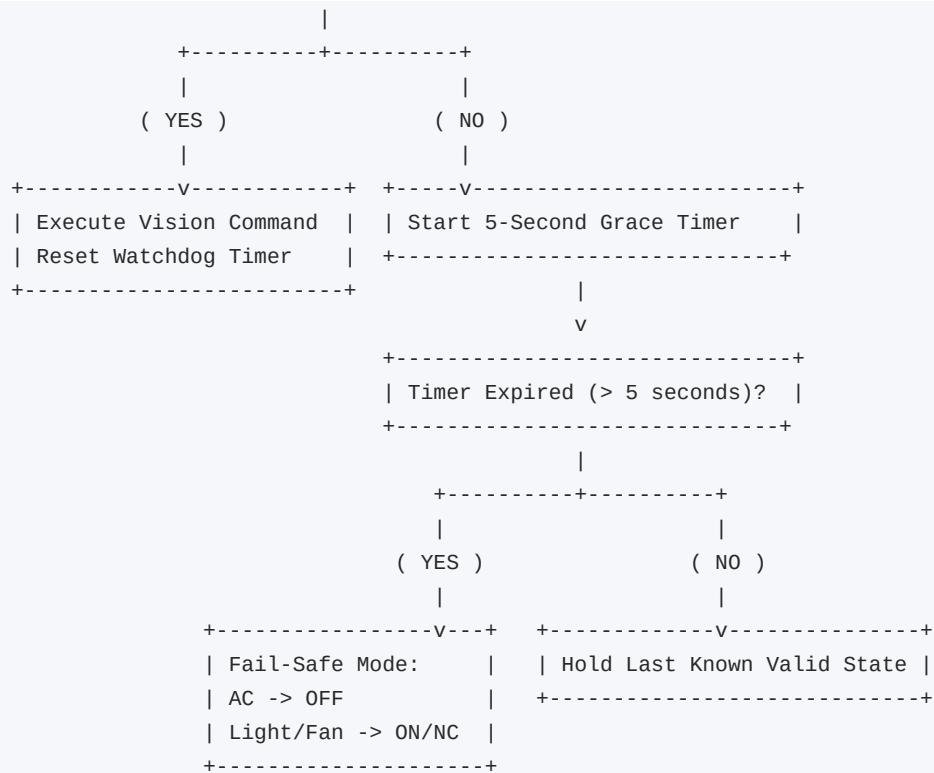
Device ID	Target Zone	Max Load	Relay Type	ESP32 GPIO	Trigger Logic	Safety Circuitry
---	---	---	---	---	---	---
light	Desk (Zone A)	40 W	5V Optocoupler Relay	GPIO 18	Active LOW	Snubber on AC output
fan	Desk (Zone A)	65 W	5V Optocoupler Relay	GPIO 19	Active LOW	Flyback diode across DC inductive motor / Snubber for AC
ac	Transit (Zone B)	1200 W	30A Solid State Relay (SSR)	GPIO 21	Active HIGH	Zero-cross switching + Heatsink

Note: GPIO Selection Note: Avoid using ESP32 strapping pins (^GPIO 0, 2, 12, 15) or input-only pins (^GPIO 34-39). Using these risks relay flickering during startup reset cycles.

4. Hardware Safety, Fail-Safe, & Edge Resiliency Rules

[Hardware Fail-Safe Logic Flow]

```
+-----+
| Is WebSocket / MQTT alive? |
+-----+
```



1. Vision Debounce & Anti-Flicker:

- Minimum occupancy hysteresis: 15 seconds.
- State transitions to `OFF` will hold for 15 seconds after zero detection to avoid power-cycling relays on temporary camera occlusion.

2. Relay Terminal Wiring (NO vs. NC):

- Lights & Fans: Wire to Normally Closed (NC) terminal so room lights default to ON if the micro-controller loses total power.
- High-Power AC: Wire to Normally Open (NO) terminal so high-current heating/cooling loads drop out safely on hardware fault.

3. Hardware Watchdog:

- Embedded MCU must run a 5-second Software Watchdog Timer (WDT). If no telemetry updates are received over WS/MQTT within 5 seconds, fallback to local manual override mode.

4. Inductive Inrush Protection:

- Motorized loads (fans/compressors) must maintain a minimum 3-minute lockout delay between consecutive restart cycles to protect AC compressor motors from back-pressure lockout.